

Laboratoire SCF

semestre de printemps 2025 - 2026

Device Tree overlay

Objectifs pédagogiques

Ce laboratoire a pour but de créer un device tree overlay permettant de reprogrammer la FPGA.

Ajout du support pour `configs`

Avant toutes choses, il faut ajouter le support des `configs` pour les overlays dans le kernel. Malheureusement, le module correspondant n'a pas été ajouté au kernel mainline et il faut donc le faire nous-même. Pour cela, le fichier `of_configs.patch` sur le repo des labos contient un patch qui permet d'ajouter le fichier `drivers/of/configs.c`, ainsi que modifier les fichiers `Kconfig` et `Makefile` dans `drivers/of`. La commande suivante permet de l'appliquer (depuis la racine du kernel) :

```
$ git apply <path_to_of_configs.patch>
```

A l'aide de `menuconfig` (ne pas oublier de mettre les options `ARCH/CROSS_COMPILE` pour `make`), il faut ensuite activer les options `OF_CONFIGS` et `OF_FPGA_REGION`.

Pour pouvoir appliquer les overlays, il faut compiler le DTS avec une option supplémentaire qui ajoute des informations au DTB. Ajouter la ligne suivante dans le fichier `Makefile` qui se trouve dans le même dossier que le DTS de la carte (adapter le nom en fonction du nom de votre fichier DTS, sans l'extension) :

```
DTC_FLAGS_socfpga_cyclone5_sockit := -@
```

Finalement, il faut lancer la compilation du kernel, régénérer le fichier `.itb` et le copier sur la carte SD dans la partition boot. Sur la carte, il est possible de copier un fichier sur la partition de boot à l'aide de :

```
$ mount /dev/mmcblk0p1 /mnt/boot  
$ cp /mnt/boot/delsoc.itb /mnt/boot/delsoc.itb.bak  
$ mv delsoc.itb /mnt/boot/delsoc.itb
```

Après redémarrage de la carte, la commande suivante permet de vérifier que le `configs` soit bien présent. Elle doit s'exécuter correctement et le sous dossier `device-tree/overlays` doit être présent :

```
$ mount -t configs none /sys/kernel/config
```

Il est également possible d'ajouter la ligne suivante dans `/etc/fstab` afin d'automatiquement monter `configs`

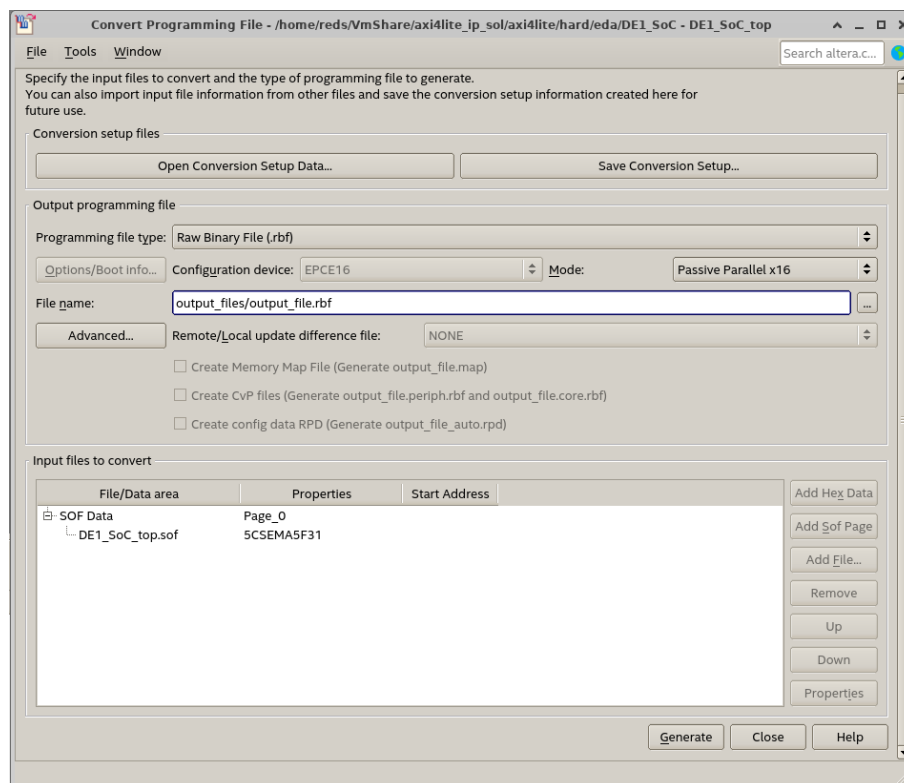
```
configs /sys/kernel/config configs defaults 0 0
```

Génération du .rbf

Les bitstreams sont des fichiers binaires qui permettent de programmer les FPGAs avec le design voulu. Avec la DE1-SoC et pour Linux, ces fichiers ont l'extension `.rbf` et il est possible de les générer depuis Quartus.

Pour ce faire, depuis le projet Quartus du design des laboratoires 5 à 7 (bien évidemment, il doit déjà être compilé), il faut aller sous `File > Convert programming File` et mettre les paramètres suivants :

1. Programming file type : Raw Binary File (.rbf)
2. Mode : Passive Parallel x16 (compatible avec la configuration des MSEL à 00000)
3. File Name : Fichier de sortie voulu (relatif au dossier `eda/`)
4. Input file to convert : Fichier `.sof` (à priori dans `output_files`, même fichier utilisé pour le script Python)
5. Cliquer sur `Generate`



Le fichier généré doit ensuite être copié sur la carte dans le dossier `/lib/firmware`.

Création d'un overlay

On souhaite avoir un overlay de device tree qui permet de :

- Charger le bitstream (`.rbf`) sur la FPGA
- Charger le noeud pour le module (avec le compatible, registre, ...).
 - Le nom du compatible peut être changé pour être différents de celui du device tree de base.
 - Si nécessaire, un driver simple est fourni qui récupère l'adresse depuis le device tree et valide la constante.

Il faut pour cela déterminer si des noeuds existent déjà avec une configuration partielle ou avec le status `disable` ou s'il faut en ajouter. Les documentations suivantes présentent les différents *bindings* pour la FPGA sur la DE1-SoC :

- `fpga-bridge`¹

1. <https://elixir.bootlin.com/linux/v6.18/source/Documentation/devicetree/bindings/fpga/fpga-bridge.yaml>

- `fpga-region`²
- `socfpga-fpga-mgr`³
- `socfpga-hps2fpga`⁴
- `socfpga-sdram2fpga`⁵
- Documentation overlay⁶

Un exemple d'overlay est disponible sur le repo Git. Ces fichiers ont l'extension `.dtso` et il est possible de les compiler avec des modules *out-of-tree* en ajoutant la ligne suivante dans le Makefile :

```
dtb-y += my_overlay.dtbo
```

Une fois compilé, le fichier `dtbo` généré doit être mis dans le dossier `/lib/firmware` sur la carte.

Charger l'overlay

Pour charger un overlay, il faut exécuter les commandes suivantes (configfs doit être monté) :

```
$ mkdir -p /sys/kernel/config/device-tree/overlays/my_fpga_region
$ echo my_overlay.dtbo > /sys/kernel/config/device-tree/overlays/my_fpga_region/path
```

Le nom `my_fpga_region` peut être changé comme voulu. `my_overlay.dtbo` doit correspondre au nom du fichier `.dtbo` mis dans `/lib/firmware`.

Les drivers implémentant des platforms devices peuvent être inséré avec l'ajout de l'overlay, le callback `probe` sera alors appelé automatiquement.

Des messages comme le suivant risquent de s'afficher lors de l'ajout de l'overlay, ils ne sont pas gênants. Tout autre message peut l'être cependant.

```
OF: overlay: WARNING: memory leak will occur if overlay removed, property: ...
```

Le dossier `/sys/firmware/devicetree/base/` contient l'architecture *finale* du device-tree de base et des différents overlay. Il permet donc de debug les différentes propriétés appliquées. Certaines propriétés (`reg`, `ranges`) sont en retournées en binaire, il est possible d'utiliser `xxd` à la place de `cat` pour les lire.

Les dossiers contenus dans `/sys/class/fpga_*` ont également des sous dossiers `of_node` avec les propriétés les concernant venant du device tree.

Le fichier `/sys/class/fpga_manager/fpga0/state` permet de voir le status de la FPGA. Etant donné que U-Boot précharge un bitstream, celui-ci devrait tout le temps être en mode `operating`. Ce n'est pas un problème, car le driver de la FPGA va automatiquement l'arrêter le temps de charger un nouveau bitstream.

Il est toute fois possible de désactiver le chargement par U-Boot en enlevant `run fpgaload;` de la variable `start` dans le fichier `uEnv.txt` sur la partition boot. Après quoi, le status devrait être `power off` (après avoir coupé le courant sur la carte) et repasser à `operating` après chargement de l'overlay.

2. <https://elixir.bootlin.com/linux/v6.18/source/Documentation/devicetree/bindings/fpga/fpga-region.yaml>

3. <https://elixir.bootlin.com/linux/v6.18/source/Documentation/devicetree/bindings/fpga/altera-socfpga-fpga-mgr.txt>

4. <https://elixir.bootlin.com/linux/v6.18/source/Documentation/devicetree/bindings/fpga/altr,socfpga-hps2fpga-bridge.yaml>

5. <https://elixir.bootlin.com/linux/v6.18/source/Documentation/devicetree/bindings/fpga/altr,socfpga-fpga2sdram-bridge.yaml>

6. <https://docs.kernel.org/6.18/devicetree/overlay-notes.html>